RIG SAFETY QHSE MONITORING SYSTEM

A Main-Project Report

Submitted for the partial fulfillment of the requirement for the award of the degree of

MASTER OF SCIENCE IN INFORMATION TECHNOLOGY

Submitted by

RIYASH KHAN A(A24MIT20)

Under the Guidance of

Dr. A. Lourdu Caroline M.C.A., M.Phil., Ph.D.,

(Assistant Professor, PG Department of Computer Applications)

PG DEPARTMENT OF COMPUTER APPLICATIONS

ST.JOSEPH'S COLLEGE OF ARTS AND SCIENCE(AUTONOMOUS)

AFFILIATED TO ANNAMALAI UNIVERSITY, CHIDAMBARAM

CUDDALORE – 607 001

APRIL-2026

CERTIFICATE

This is to certify that the Main-Project entitled

RIG SAFETY QHSE MONITORING SYSTEM

Being submitted to

ST.JOSEPH'S COLLEGE OF ARTS AND SCIENCE(AUTONOMOUS)

(Affiliated to Annamalai university-Chidambaram)

Submitted By

RIYASH KHAN A (A24MIT20)

For the partial Fulfillment for the award of degree of

MASTER OF SCIENCE IN INFORMATION TECHNOLOGY

Is a Bonafide record of work carried out by him, Under my guidance and supervision.

INTERNAL GUIDE HEAD OF THE DEPARTMENT

Submitted for the viva-voce Examination on_____

Examiners:

1. _____

2. _____

ACKNOWLEDGEMENT

First and foremost, I thank God, the Almighty, for showering his kindness and blessings to complete this project successfully.

I extend my sincere gratitude to respected **Rev. Fr. Dr. M. Swaminathan,** Secretary and **Dr. M. Arumai Selvam,** Principal, St.Joseph's College of Arts & Science(Autonomous), Cuddalore for providing such a good congenial environment to enlighten my knowledge.

I extend my hearty gratitude to **Dr. A. John Pradeep Ebenezer,** Head, PG Department of Computer Applications for the moral support and encouragement during the project.

I extend my warm gratitude to my guide **Dr. A. Lourdu Caroline,** Assistant professor of PG Department of Computer Applications for the able guidance during the project.

At the outset I take pleasure to extend my sincere gratitude to all my teachers who had taken pains to shape and model me in all my endeavours. I bound to express my sincere and hearty gratitude to my dear parents for their financial and material support which helped me to make this project a successful one. Last but not the least I place my deepest sense of gratitude to all my friends who has supported me to complete this project more successfully.

RIYASH KHAN A
(A24MIT20)

TABLE CONTENT

ABSTRACT

The QHSE (Quality, Health, Safety, and Environment) Dashboard Backend is a system designed to manage and analyze Quality, Health, Safety, and Environment data within an organization. It is developed using FastAPI and provides RESTful APIs for handling modules such as incidents, inspections, and corrective actions. The system processes key safety metrics like TRIR and LTIFR, helping organizations monitor performance and identify risks. Its modular design ensures scalability and efficient data management. Overall, the project supports better decision-making, improves workplace safety, and ensures regulatory compliance.

PROJECT DESCRIPTION

Project Description:

The QHSE (Quality, Health, Safety, and Environment) Dashboard Backend is a scalable server-side application designed to manage, process, and deliver safety-related data for organizational monitoring and decision-making. The system acts as the core engine behind a dashboard that visualizes key safety and compliance metrics.

Developed using modern backend frameworks such as FastAPI, the application provides RESTful APIs for handling various QHSE data entities, including incidents, inspections, audits, corrective actions, and behavioral observations. It integrates with a relational database using ORM tools like SQLAlchemy to ensure efficient data storage, retrieval, and manipulation.

The backend is structured using a modular architecture, separating concerns into routes, services, and data models. This design improves maintainability, scalability, and code readability. It supports the calculation and aggregation of critical metrics such as Total Recordable Incident Rate (TRIR), Lost Time Injury Frequency Rate (LTIFR), near-miss tracking, and year-to-date safety statistics.

Additionally, the system enables real-time data access for dashboards, allowing organizations to monitor trends, identify risks, and take proactive safety measures. Error handling, validation, and API documentation are incorporated to ensure reliability and ease of integration with frontend systems.

Overall, the project provides a strong backend foundation for building intelligent QHSE dashboards, helping organizations enhance safety performance, ensure regulatory compliance, and promote a culture of continuous improvement.

HARDWARE AND SOFTWARE REQUIREMENT

HARDWARE REQUIREMENT:

- SYSTEM : Pentium IV 2.4GHz
- HARDDISK : 500GB
- FLOPPY DRIVE : 1.44MB
- MONITOR : 15VGA color
- MOUSE : Logitech
- RAM : 8GB
- KEYBOARD : 110 keys enhanced

SOFTWARE REQUIREMENTS

- OPERATING SYSTEM : Windows11 Professional
- PROCESSOR : INTELI512THGEN64BIT
- FRONTEND : TYPESCRIPT, REACT
- BACKEND : PYTHON
- RAM : 8GB

SOFTWARE DESCRIPTION

1. Overview

The system is a full-stack web application built using a modern, decoupled architecture. It consists of:

- A frontend client implemented in TypeScript and React
- A backend service developed in Python
- A relational database powered by SQLite

The architecture follows a RESTful client-server model, enabling clear separation of concerns, scalability, and maintainability.

2. Frontend (TypeScript + React)

The frontend is responsible for presentation logic, user interaction, and state management.

Core Characteristics

- TypeScript Integration
  - Enforces static typing for improved code quality and maintainability
  - Reduces runtime errors via compile-time checks
- React Framework
  - Component-based architecture for reusable UI elements
  - Virtual DOM for efficient rendering
  - Hooks (e.g., useState, useEffect) for state and lifecycle management

Functional Responsibilities

- Rendering dynamic UI components
- Handling user inputs and validation
- Managing application state (local or via libraries like Redux/Zustand)
- Making asynchronous API calls to the backend
- Displaying processed data from the server

Communication Layer

- Uses HTTP/HTTPS to interact with backend APIs
- Typically employs fetch or axios for requests

- Data format: JSON

---

3. Backend (Python)

The backend acts as the application core, managing business logic and data operations.

Framework Options

- Lightweight frameworks such as:
    - Flask (minimalistic, flexible)
    - FastAPI (high performance, async support, automatic docs)

Core Responsibilities

- Exposing RESTful API endpoints
- Processing client requests
- Implementing business rules and validation
- Managing authentication/authorization (if applicable)
- Interfacing with the SQLite database

API Design

- Follows REST principles:
    - GET → retrieve data
    - POST → create records
    - PUT/PATCH → update records
    - DELETE → remove records

Example Endpoint Structure

/api/users

/api/tasks

/api/auth/login

---

4. Database (SQLite)

SQLite is used as a lightweight, embedded relational database.

Key Characteristics

- Serverless (no separate DB server required)

- File-based storage
- ACID-compliant transactions
- Ideal for small to medium-scale applications

Schema Design

- Structured using relational tables
- Common entities:
    - Users
    - Records / Tasks / Items
- Supports:
    - Primary keys
    - Foreign keys
    - Indexing for performance

Example Table

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    created_ at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  );
```

Data Flow

1. User interacts with UI (e.g., submits form)
2. React sends request to backend API
3. Backend processes request and interacts with database
4. SQLite returns query results
5. Backend formats response (JSON)
6. Frontend updates UI dynamically

SOURCE CODE

MAIN .TSX

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

APP.TSX

```
import React from 'react';
import { ThemeProvider } from '@mui/material/styles';
import CssBaseline from '@mui/material/CssBaseline';
import {  BrowserRouter as Router,
  Routes,
  Route,
  Link as RouterLink,
  useLocation,
  Navigate,
  useNavigate,
} from 'react-router-dom';
import {
  Box,
  AppBar,
  Toolbar,
```

```
  Typography,
  Drawer,
  List,
  ListItem,
  ListItemButton,
  ListItemIcon,
  ListItemText,
  IconButton,
  Divider,
  Button,
} from '@mui/material';
import {
  InsertChart as ChartIcon,
  Menu as MenuIcon,
  FiberManualRecord as DotIcon,
  Wysiwyg as ObservationIcon,
  FactCheck as MVVIcon,
  Warning as IncidentIcon,
  Assignment as InspectionIcon,
  Download as DownloadIcon,
  Logout as LogoutIcon,
  AssignmentTurnedIn as CorrectiveIcon,
} from '@mui/icons-material';

import theme from './theme';
import WeeklyChartDashboard from './components/WeeklyChartDashboard';
import NMOBB from './components/Tabs/NMOBB';
import MVV from './components/Tabs/MVV';
import Insident from './components/Tabs/Insident';
import Inspection from './components/Tabs/Inspection';
import DownloadReports from './components/Tabs/DownloadReports';
import sinopecLogo from './assets/sinopec.png';
```

```
import Corrective from './components/Tabs/Corrective';
import Login from './components/Login';
import { AuthProvider, useAuth } from './context/AuthContext';

const drawerWidth = 248;

const NavItem: React.FC<{
  text: string;
  icon: React.ReactNode;
  path: string;
  onClose: () => void;
}> = ({ text, icon, path, onClose }) => {
  const location = useLocation();
  const isActive = location.pathname === path;

  return (
    <ListItem disablePadding>
      <ListItemButton
        component={RouterLink}
        to={path}
        onClick={onClose}
        selected={isActive}
      >
        <ListItemIcon sx={{ color: isActive ? '#1565C0' : '#90A4AE' }}>
          {icon}
        </ListItemIcon>
        <ListItemText primary={text} />
      </ListItemButton>
    </ListItem>
  );
};
```

```
const DrawerContent: React.FC<{ onClose: () => void }> = ({ onClose }) => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const menuItems = [
    { text: 'Dashboard',        icon: <ChartIcon fontSize="small" />,       path: '/' },
    { text: 'NMOBB',            icon: <ObservationIcon fontSize="small" />, path: '/nmobb' },
    { text: 'MVV',              icon: <MVVIcon fontSize="small" />,         path: '/mvv' },
    { text: 'Incident',         icon: <IncidentIcon fontSize="small" />,    path: '/incident' },
    { text: 'Inspection',       icon: <InspectionIcon fontSize="small" />,  path: '/inspection' },
    { text: 'Corrective Actions', icon: <CorrectiveIcon fontSize="small" />, path: '/corrective' },
    { text: 'Download Reports', icon: <DownloadIcon fontSize="small" />,    path: '/download-
reports' },
  ];

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', height: '100%' }}>
      {/* Blue top accent stripe */}
      <Box sx={{ height: 4, background: 'linear-gradient(90deg, #0D47A1 0%, #1976D2 60%,
#29B6F6 100%)' }} />

      {/* Logo / Brand */}
      <Box
        sx={{
          px: 2,
          py: 2,
          display: 'flex',
          flexDirection: 'column',
```

```
    alignItems: 'center',
    gap: 1,
    background: 'linear-gradient(180deg, rgba(21,101,192,0.04) 0%, transparent 100%)',
    borderBottom: '1px solid rgba(21,101,192,0.08)',
  }}
>
  <img
    src={sinopecLogo}
    alt="Sinopec Logo"
    style={{
      height: 64,
      width: 'auto',
      objectFit: 'contain',
    }}
  />
</Box>

{/* Nav links */}
<List sx={{ px: 0.5, flexGrow: 1 }}>
  {menuItems.map((item) => (
    <NavItem key={item.text} {...item} onClose={onClose} />
  ))}
</List>

{/* Status footer */}
<Divider />
<Box
  sx={{
    px: 2.5,
    py: 1.5,
    display: 'flex',
    alignItems: 'center',
```

```
          gap: 1,
          background: 'rgba(21,101,192,0.02)',
        }}
      >
        <DotIcon
          sx={{
            fontSize: 10,
            color: '#2E7D32',
            filter: 'drop-shadow(0 0 4px rgba(46,125,50,0.6))',
            animation: 'pulse 2.5s ease-in-out infinite',
            '@keyframes pulse': {
              '0%, 100%': { opacity: 1 },
              '50%': { opacity: 0.45 },
            },
          }}
        />
        <Typography variant="caption" sx={{ color: '#90A4AE', ml: 'auto' }}>
          {user?.username || 'Admin'}
        </Typography>
        <Button
          size="small"
          startIcon={<LogoutIcon />}
          onClick={handleLogout}
          sx={{ color: '#90A4AE', fontSize: '0.7rem' }}
        >
          Logout
        </Button>
      </Box>
    </Box>
  );
};
```

```
/* ── AppBar height token – single source of truth ──────────── */
const APP_BAR_HEIGHT = 96; // px — adjust freely here

// Protected Route Component
const ProtectedRoute: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const { isAuthenticated } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  return <>{children}</>;
};

// AppContent component - protected layout with sidebar
const AppContent: React.FC = () => {
  const [mobileOpen, setMobileOpen] = React.useState(false);

  return (
    <Box sx={{ display: 'flex', minHeight: '100vh' }}>
      {/* ── AppBar ──────────────────────────────────────────────── */}
      <AppBar
        position="fixed"
        elevation={2}
        sx={{
          width: { sm: `calc(100% - ${drawerWidth}px)` },
          ml: { sm: `${drawerWidth}px` },
          height: APP_BAR_HEIGHT,
          justifyContent: 'center',
          background: 'linear-gradient(135deg, #0D47A1 0%, #1565C0 55%, #1976D2 100%)',
        }}
```

```
>
  <Toolbar
    disableGutters
    sx={{
      px: { xs: 2, sm: 3 },
      height: APP_BAR_HEIGHT,
      minHeight: `${APP_BAR_HEIGHT}px !important`,
      display: 'flex',
      alignItems: 'center',
      gap: 1,
    }}
  >
    {/* Mobile hamburger */}
    <IconButton
      color="inherit"
      edge="start"
      onClick={() => setMobileOpen((p) => !p)}
      sx={{ mr: 1, display: { sm: 'none' } }}
    >
      <MenuIcon />
    </IconButton>

    {/* Centred header text block */}
    <Box
      sx={{
        flexGrow: 1,
        display: 'flex',
        flexDirection: 'column',
        alignItems: 'center',
        justifyContent: 'center',
        gap: 0.25,
      }}
```

```
>
  {/* Line 1 – company name */}
  <Typography
    sx={{
      fontWeight: 700,
      fontSize: { xs: '0.68rem', sm: '0.78rem', md: '0.85rem' },
      letterSpacing: '0.06em',
      textTransform: 'uppercase',
      color: '#E3F2FD',
      textAlign: 'center',
      lineHeight: 1.3,
    }}
  >
    Branch of Sinopec International Petroleum Service Corporation – K.S.A
  </Typography>

  {/* Line 2 – department */}
  <Typography
    sx={{
      fontWeight: 600,
      fontSize: { xs: '0.68rem', sm: '0.78rem', md: '0.85rem' },
      letterSpacing: '0.05em',
      textTransform: 'uppercase',
      color: '#BBDEFB',
      textAlign: 'center',
      lineHeight: 1.3,
    }}
  >
    HSE Department – Drilling & Workover Operation
  </Typography>

  {/* Line 3 – product name (largest) */}
```

```
      <Typography
        sx={{
          fontWeight: 800,
          fontSize: { xs: '1rem', sm: '1.2rem', md: '1.35rem' },
          letterSpacing: '0.08em',
          textTransform: 'uppercase',
          color: '#FFFFFF',
          textAlign: 'center',
          lineHeight: 1.4,
          textShadow: '0 1px 4px rgba(0,0,0,0.25)',
          mt: 0.25,
        }}
      >
        Rig Safety QHSE System
      </Typography>
    </Box>
  </Toolbar>

  {/* Subtle bottom divider */}
  <Box sx={{ height: 1, background: 'rgba(255,255,255,0.12)' }} />
</AppBar>

{/* ── Sidebar ──────────────────────────────────────────────────────── */}
<Box
  component="nav"
  sx={{ width: { sm: drawerWidth }, flexShrink: { sm: 0 } }}
>
  {/* Mobile drawer */}
  <Drawer
    variant="temporary"
    open={mobileOpen}
    onClose={() => setMobileOpen(false)}
```

```
    ModalProps={{ keepMounted: true }}
    sx={{
      display: { xs: 'block', sm: 'none' },
      '& .MuiDrawer-paper': { boxSizing: 'border-box', width: drawerWidth },
    }}
  >
    <DrawerContent onClose={() => setMobileOpen(false)} />
  </Drawer>

  {/* Desktop permanent drawer */}
  <Drawer
    variant="permanent"
    open
    sx={{
      display: { xs: 'none', sm: 'block' },
      '& .MuiDrawer-paper': { boxSizing: 'border-box', width: drawerWidth },
    }}
  >
    <DrawerContent onClose={() => {}} />
  </Drawer>
</Box>

{/* ── Main content ──────────────────────────────────────────── */}
<Box
  component="main"
  sx={{
    flexGrow: 1,
    p: 3,
    width: { sm: `calc(100% - ${drawerWidth}px)` },
    minHeight: '100vh',
    // Push content below the fixed AppBar
    mt: `${APP_BAR_HEIGHT}px`,
```

```
      }}
    >
      <Routes>
        <Route path="/"              element={<WeeklyChartDashboard />} />
        <Route path="/nmobb"         element={<NMOBB />} />
        <Route path="/mvv"           element={<MVV />} />
        <Route path="/inspection"    element={<Inspection />} />
        <Route path="/incident"      element={<Insident />} />
        <Route path="/download-reports" element={<DownloadReports />} />
        <Route path="/corrective"    element={<Corrective />} />
      </Routes>
    </Box>
  </Box>
 );
};

// Main App component with routes
const App: React.FC = () => {
 return (
  <ThemeProvider theme={theme}>
   <CssBaseline />
   <AuthProvider>
    <Router>
     <Routes>
      {/* Public Routes */}
      <Route path="/login" element={<Login />} />

      {/* Protected Routes */}
      <Route
       path="/*"
       element={
        <ProtectedRoute>
```

```
                <AppContent />
              </ProtectedRoute>
            }
          />
        </Routes>
      </Router>
    </AuthProvider>
  </ThemeProvider>
 );
};

export default App;
```

INCIDENT.TSX

```
import React, { useState } from "react";
import {
 Card,
 Form,
 Input,
 InputNumber,
 Button,
 Row,
 Col,
 ConfigProvider,
 DatePicker,
 Typography,
 Divider,
 Tabs,
```

```
  message,
} from "antd";
import dayjs from "dayjs";
import isoWeek from "dayjs/plugin/isoWeek";
import weekOfYear from "dayjs/plugin/weekOfYear";
import advancedFormat from "dayjs/plugin/advancedFormat";
import "antd/dist/reset.css";
import { incidentAPI, IncidentRecord } from "../../services/incidentAPI";
import { ltiFreeAPI, LTIFreeRecord } from "../../services/ltiFreeAPI";

dayjs.extend(isoWeek);
dayjs.extend(weekOfYear);
dayjs.extend(advancedFormat);

const { Title, Text } = Typography;
const { WeekPicker } = DatePicker;
const { TextArea } = Input;

/* ── Theme ──────────────────────────────────────────────────────────────────
*/
const antdTheme = {
  token: {
    colorPrimary: "#1565C0",
    colorBorder: "rgba(21,101,192,0.2)",
    borderRadius: 8,
    fontFamily: '"Inter", "Helvetica Neue", sans-serif',
    colorText: "#0D1F3C",
    colorBgContainer: "#FFFFFF",
    controlHeight: 36,
    fontSize: 14,
  },
};
```

```
/* ── Lagging Indicators ──────────────────────────────────────────────── */
const LAGGING_INDICATORS: { label: string; name: string }[] = [
  { label: "LTI – Lost Time Injury", name: "lti" },
  { label: "RDI – Restricted Duty Injury", name: "rdi" },
  { label: "MTC – Medical Treatment Case", name: "mtc" },
  { label: "First Aid / FAI – First Aid Injury", name: "fai" },
  { label: "Near Miss", name: "nearMiss" },
  { label: "Fire", name: "fire" },
  { label: "MVA – Motor Vehicle Accident", name: "mva" },
  { label: "Oil Spill / SPILL", name: "spill" },
  { label: "Property Damage / PD", name: "propertyDamage" },
  { label: "NWR – Non Work Related", name: "nwr" },
  { label: "FAT – Fatalities", name: "fat" },
  { label: "Drop Object", name: "dropObject" },
  { label: "Off Job Incident", name: "offJobIncident" },
];

/* ── Inline styles ──────────────────────────────────────────────────── */
const styles: Record<string, React.CSSProperties> = {
  page: {
    padding: "clamp(12px, 4vw, 32px)",
    maxWidth: 1100,
    margin: "0 auto",
    boxSizing: "border-box" as const,
  },
  headerWrap: { marginBottom: 4 },
  card: {
    marginTop: 20,
    borderRadius: 12,
    boxShadow: "0 2px 12px rgba(21,101,192,0.08)",
  },
```

```
  sectionTitle: {
    fontSize: "clamp(13px, 2vw, 15px)",
    fontWeight: 600,
    color: "#1565C0",
    marginBottom: 8,
    marginTop: 4,
    textTransform: "uppercase" as const,
    letterSpacing: "0.05em",
  },
  divider: { borderColor: "rgba(21,101,192,0.12)", margin: "12px 0" },
  actionRow: {
    display: "flex",
    flexWrap: "wrap" as const,
    justifyContent: "flex-end",
    gap: 10,
    marginTop: 16,
  },
  btn: { minWidth: 120, height: 44, fontSize: 15, flex: "0 0 auto" },
};

const SectionHeader: React.FC<{ children: React.ReactNode }> = ({ children }) => (
  <div style={styles.sectionTitle}>{children}</div>
);

/* ── Incident Report Form ──────────────────────────────────────────── */
const IncidentForm: React.FC = () => {
  const [form] = Form.useForm();
  const [loading, setLoading] = useState(false);

  const onFinish = async (values: any) => {
    setLoading(true);
    try {
```

```
    // Transform field names to match backend schema
    const payload: IncidentRecord = {
      rig_number: values.rigNumber,
      week: values.week ? values.week.format("YYYY-MM-DD") : "",
      lti: values.lti || 0,
      rdi: values.rdi || 0,
      mtc: values.mtc || 0,
      fai: values.fai || 0,
      near_miss: values.nearMiss || 0,
      fire: values.fire || 0,
      mva: values.mva || 0,
      spill: values.spill || 0,
      property_damage: values.propertyDamage || 0,
      nwr: values.nwr || 0,
      fat: values.fat || 0,
      drop_object: values.dropObject || 0,
      off_job_incident: values.offJobIncident || 0,
      manhours: values.manhours || 0,
      incident_description: values.incidentDescription || "",
    };
    await incidentAPI.create(payload);
    message.success("Incident record submitted successfully!");
    form.resetFields();
  } catch (error) {
    message.error("Failed to submit incident record.");
  } finally {
    setLoading(false);
  }
};

return (
  <Form layout="vertical" form={form} onFinish={onFinish}>
```

```
{/* Basic Info */}
<Row gutter={[8, 8]}>
  <Col xs={24} sm={24} md={12} lg={12}>
    <Form.Item
      label="Rig Number"
      name="rigNumber"
      rules={[{ required: true, message: "Enter Rig Number" }]}
    >
      <Input placeholder="e.g. SP-103" size="large" />
    </Form.Item>
  </Col>
  <Col xs={24} sm={24} md={12} lg={12}>
    <Form.Item
      label="Select Week"
      name="week"
      rules={[{ required: true, message: "Select Week" }]}
    >
      <WeekPicker style={{ width: "100%" }} size="large" />
    </Form.Item>
  </Col>
</Row>

<Divider style={styles.divider} />

{/* Incident Count – Lagging Indicators */}
<SectionHeader>Incident Count — Lagging Indicators</SectionHeader>
<Row gutter={[8, 8]}>
  {LAGGING_INDICATORS.map(({ label, name }) => (
    <Col key={name} xs={24} sm={12} md={8} lg={8}>
      <Form.Item label={label} name={name}>
        <InputNumber min={0} placeholder="0" size="large" style={{ width: "100%" }} />
      </Form.Item>
```

```
          </Col>
        ))}
      </Row>

      <Divider style={styles.divider} />

      {/* Exposure Data */}
      <SectionHeader>Exposure Data</SectionHeader>
      <Row gutter={[8, 8]}>
        <Col xs={24} sm={24} md={12} lg={12}>
          <Form.Item
            label="Manhours"
            name="manhours"
            rules={[{ required: true, message: "Enter Manhours" }]}
          >
            <InputNumber min={0} placeholder="e.g. 1200" size="large" style={{ width: "100%" }}
/>
          </Form.Item>
        </Col>
      </Row>

      <Divider style={styles.divider} />

      {/* Weekly Summary */}
      <SectionHeader>Weekly Summary</SectionHeader>
      <Row gutter={[8, 8]}>
        <Col xs={24}>
          <Form.Item label="Incident Description" name="incidentDescription">
            <TextArea rows={3} placeholder="Enter Description" size="large" style={{ resize:
"vertical" }} />
          </Form.Item>
        </Col>
```

```
      </Row>

      <div style={styles.actionRow}>
        <Button style={styles.btn} onClick={() => form.resetFields()}>Reset</Button>
        <Button type="primary" htmlType="submit" style={styles.btn} loading={loading}>
          Submit Incident
        </Button>
      </div>
    </Form>
  );
};

/* ── LTI Free Count Form ──────────────────────────────────────────────── */
const LtiFreeCountForm: React.FC = () => {
  const [form] = Form.useForm();
  const [loading, setLoading] = useState(false);

  const onFinish = async (values: any) => {
    setLoading(true);
    try {
      const payload: LTIFreeRecord = {
        rig_number: values.rigNumber,
        starting_date: values.startingDateWithoutLti ?
values.startingDateWithoutLti.format("YYYY-MM-DD") : "",
        commencement_date: values.commencementDate ?
values.commencementDate.format("YYYY-MM-DD") : "",
      };
      await ltiFreeAPI.create(payload);
      message.success("LTI Free record submitted successfully!");
      form.resetFields();
    } catch (error) {
      message.error("Failed to submit LTI Free record.");
```

```
  } finally {
    setLoading(false);
  }
 };

 return (
  <Form layout="vertical" form={form} onFinish={onFinish}>
    <SectionHeader>LTI Free Count Details</SectionHeader>
    <Row gutter={[8, 8]}>
      <Col xs={24} sm={24} md={12} lg={12}>
        <Form.Item
          label="Rig Number"
          name="rigNumber"
          rules={[{ required: true, message: "Enter Rig Number" }]}
        >
          <Input placeholder="e.g. SP-103" size="large" />
        </Form.Item>
      </Col>
      <Col xs={24} sm={24} md={12} lg={12}>
        <Form.Item
          label="Starting Date Without LTI"
          name="startingDateWithoutLti"
          rules={[{ required: true, message: "Select starting date without LTI" }]}
        >
          <DatePicker style={{ width: "100%" }} size="large" placeholder="Select date"
format="DD-MM-YYYY" />
        </Form.Item>
      </Col>
      <Col xs={24} sm={24} md={12} lg={12}>
        <Form.Item
          label="Commencement Date"
          name="commencementDate"
```

```
          rules={[{ required: true, message: "Select commencement date" }]}
        >
          <DatePicker style={{ width: "100%" }} size="large" placeholder="Select date"
format="DD-MM-YYYY" />
        </Form.Item>
      </Col>
    </Row>
    <div style={styles.actionRow}>
      <Button style={styles.btn} onClick={() => form.resetFields()}>Reset</Button>
      <Button type="primary" htmlType="submit" style={styles.btn} loading={loading}>
        Submit LTI Free Count
      </Button>
    </div>
  </Form>
 );
};

/* ── Tab Items ──────────────────────────────────────────────────────────
*/
const tabItems = [
 { key: "1", label: "Incident Report", children: <IncidentForm /> },
 { key: "2", label: "LTI Free Count", children: <LtiFreeCountForm /> },
];

/* ── Root
──────────────────────────────────────────────────────────────────────── */
const Incident: React.FC = () => (
 <ConfigProvider theme={antdTheme}>
  <style>{`
    @media (max-width: 767px) {
      .ant-tabs-tab { padding: 10px 12px !important; font-size: 14px; }
    }
```

```
      @media (min-width: 600px) and (max-width: 1024px) {
        .ant-select-selector,
        .ant-input,
        .ant-input-number,
        .ant-picker { min-height: 38px !important; font-size: 14px !important; }
        .ant-form-item-label > label { font-size: 13px !important; }
        .ant-tabs-tab { padding: 12px 20px !important; font-size: 15px; }
      }
      .ant-picker { width: 100% !important; }
      .ant-input-number { width: 100% !important; }
    `}</style>

    <div style={styles.page}>
      <div style={styles.headerWrap}>
        <Title level={3} style={{ marginBottom: 4, fontSize: "clamp(18px, 3vw, 24px)", textAlign:
"center" }}>
          Incident Report Entry
        </Title>
        <Text type="secondary" style={{ fontSize: "clamp(12px, 2vw, 14px)", display: "block",
marginBottom: 0, textAlign: "center" }}>
          Weekly incident data, lagging indicators, exposure hours, summary and LTI free tracking
        </Text>
      </div>

      <Card style={styles.card}>
        <Tabs defaultActiveKey="1" items={tabItems} size="large" tabBarStyle={{
marginBottom: 24 }} />
      </Card>
    </div>
  </ConfigProvider>
);
```

```
export default Incident;
```

INSPECTION.TSX

```
import React, { useState } from "react";
import {
  Card,
  Form,
  Select,
  Input,
  InputNumber,
  Button,
  Row,
  Col,
  ConfigProvider,
  DatePicker,
  Typography,
  message,
} from "antd";
import dayjs from "dayjs";
import "antd/dist/reset.css";
import { inspectionAPI, InspectionRecord } from "../../services/inspectionAPI";

const { Title, Text } = Typography;
const { Option } = Select;
const { RangePicker } = DatePicker;

/* ── Theme ──────────────────────────────────────────────────────────────────
*/
const antdTheme = {
  token: {
    colorPrimary: "#1565C0",
```

```
    colorBorder: "rgba(21,101,192,0.2)",
    borderRadius: 8,
    fontFamily: '"Inter", "Helvetica Neue", sans-serif',
    colorText: "#0D1F3C",
    colorBgContainer: "#FFFFFF",
    controlHeight: 36,
    fontSize: 14,
  },
};

/* ── Data ──────────────────────────────────────────────────────────────── */
const RIGS = ["Rig 1", "Rig 2", "Rig 3", "Rig 4", "Rig 5"];
const INSPECTION_TYPES = ["RSI", "QSI", "WCI", "LPD Visits", "Div Internal"];
const PRIORITIES = ["P1", "P2", "P3", "P4"];
const STATUSES = ["Open", "Closed", "In Progress", "Rejected"];

/* ── Inspection Form ──────────────────────────────────────────────────── */
const InspectionForm: React.FC = () => {
  const [form] = Form.useForm();
  const [loading, setLoading] = useState(false);

  const onFinish = async (values: any) => {
    setLoading(true);
    try {
      const payload: InspectionRecord = {
        rig: values.rig,
        inspection_type: values.inspectionType,
        inspection_value: values.inspectionValue,
        priority: values.priority,
        target_date: values.targetDate ? values.targetDate.format("YYYY-MM-DD") : "",
        date_opened: values.dateOpened ? values.dateOpened.format("YYYY-MM-DD") : "",
```

```
        status: values.status,
        remarks: values.remarks || null,
      };
      console.log("Submitting inspection:", payload);
      const response = await inspectionAPI.create(payload);
      console.log("Success:", response);
      message.success("Inspection record submitted successfully!");
      form.resetFields();
    } catch (error: any) {
      console.error("Inspection submission error:", error);
      if (error.response) {
        // Server responded with error
        message.error(`Submission failed: ${error.response.data?.detail ||
error.response.statusText}`);
      } else if (error.request) {
        message.error("No response from server. Is the backend running?");
      } else {
        message.error("Error: " + error.message);
      }
    } finally {
      setLoading(false);
    }
  };

  return (
    <Form layout="vertical" form={form} onFinish={onFinish}>
      <Row gutter={[16, 16]}>
        {/* Rig */}
        <Col xs={24} sm={24} md={8} lg={8}>
          <Form.Item
            label="Rig"
            name="rig"
```

```
    rules={[{ required: true, message: "Please enter rig name" }]}
  >
    <Input placeholder="Enter Rig name" size="large" />
  </Form.Item>
</Col>

{/* Inspection Type */}
<Col xs={24} sm={24} md={8} lg={8}>
  <Form.Item
    label="Inspection Type"
    name="inspectionType"
    rules={[{ required: true, message: "Please select inspection type" }]}
  >
    <Select placeholder="Select Inspection Type" size="large" showSearch>
      {INSPECTION_TYPES.map((type) => (
        <Option key={type} value={type}>
          {type}
        </Option>
      ))}
    </Select>
  </Form.Item>
</Col>

{/* Inspection Value */}
<Col xs={24} sm={24} md={8} lg={8}>
  <Form.Item
    label="Inspection Value"
    name="inspectionValue"
    rules={[
      { required: true, message: "Please enter inspection value" },
      { type: "number", min: 0, message: "Value must be 0 or greater" },
    ]}
```

```
  >
    <InputNumber
      min={0}
      precision={0}
      placeholder="Enter inspection value"
      size="large"
      style={{ width: "100%" }}
    />
  </Form.Item>
</Col>

{/* Priority */}
<Col xs={24} sm={24} md={12} lg={12}>
  <Form.Item
    label="Priority"
    name="priority"
    rules={[{ required: true, message: "Please select priority" }]}
  >
    <Select placeholder="Select Priority" size="large">
      {PRIORITIES.map((priority) => (
        <Option key={priority} value={priority}>
          {priority}
        </Option>
      ))}
    </Select>
  </Form.Item>
</Col>

{/* Target Date */}
<Col xs={24} sm={24} md={12} lg={12}>
  <Form.Item
    label="Target Date"
```

```
    name="targetDate"
    rules={[{ required: true, message: "Please select target date" }]}
  >
    <DatePicker
      style={{ width: "100%" }}
      size="large"
      placeholder="Select Target Date"
      format="YYYY-MM-DD"
    />
  </Form.Item>
</Col>

{/* Date Opened */}
<Col xs={24} sm={24} md={12} lg={12}>
  <Form.Item
    label="Date Opened"
    name="dateOpened"
    rules={[{ required: true, message: "Please select date opened" }]}
  >
    <DatePicker
      style={{ width: "100%" }}
      size="large"
      placeholder="Select Date Opened"
      format="YYYY-MM-DD"
    />
  </Form.Item>
</Col>

{/* Status */}
<Col xs={24} sm={24} md={12} lg={12}>
  <Form.Item
    label="Status"
```

```
      name="status"
      rules={[{ required: true, message: "Please select status" }]}
    >
      <Select placeholder="Select Status" size="large">
        {STATUSES.map((status) => (
          <Option key={status} value={status}>
            {status}
          </Option>
        ))}
      </Select>
    </Form.Item>
  </Col>

  {/* Remarks */}
  <Col xs={24} sm={24} md={24} lg={24}>
    <Form.Item label="Remarks" name="remarks">
      <Input.TextArea rows={2} placeholder="Enter remarks" size="large" />
    </Form.Item>
  </Col>
</Row>

{/* Actions */}
<div
  style={{
    display: "flex",
    flexWrap: "wrap",
    justifyContent: "flex-end",
    gap: 10,
    marginTop: 20,
  }}
>
  <Button
```

```
          style={{ minWidth: 120, height: 44, fontSize: 15 }}
          onClick={() => form.resetFields()}
        >
          Reset
        </Button>
        <Button
          type="primary"
          htmlType="submit"
          style={{ minWidth: 120, height: 44, fontSize: 15 }}
          loading={loading}
        >
          Submit Inspection
        </Button>
      </div>
    </Form>
  );
};

/* ── Root ──────────────────────────────────────────────────────────────────── */
const Inspection: React.FC = () => (
  <ConfigProvider theme={antdTheme}>
    <style>{`
      .ant-picker { width: 100% !important; }
      .ant-input, .ant-input-number, .ant-select-selector { width: 100% !important; }
      @media (min-width: 600px) and (max-width: 1024px) {
        .ant-select-selector,
        .ant-input,
        .ant-input-number,
        .ant-picker { min-height: 38px !important; font-size: 14px !important; }
        .ant-form-item-label > label { font-size: 13px !important; }
      }
```

```
`}</style>

<div
  style={{
    padding: "clamp(12px, 4vw, 32px)",
    maxWidth: 960,
    margin: "0 auto",
    boxSizing: "border-box",
  }}
>
  <Title
    level={3}
    style={{ marginBottom: 4, fontSize: "clamp(18px, 3vw, 24px)", textAlign: "center" }}
  >
    Inspection Entry
  </Title>
  <Text
    type="secondary"
    style={{
      display: "block",
      marginBottom: 0,
      fontSize: "clamp(12px, 2vw, 14px)",
      textAlign: "center",
    }}
  >
    Submit inspection records
  </Text>

  <Card
    style={{
      marginTop: 20,
      borderRadius: 12,
```

```
            boxShadow: "0 2px 12px rgba(21,101,192,0.08)",
          }}
        >
          <InspectionForm />
        </Card>
      </div>
    </ConfigProvider>
  );

  export default Inspection;
```

MAIN.PY

```
  import uvicorn
  from fastapi import FastAPI
  from fastapi.logger import logger
  from contextlib import asynccontextmanager
  from fastapi.middleware.cors import CORSMiddleware
  from utils.scheduler import scheduler
  from database import connection
  from api.qhse_routes import qhse_router
  from api.mvv import router as mvv_router
  from api.nmobb import router as nmobb_router
  from api.incident import router as incident_router
  from api.lti_free import router as lti_free_router
  from api.inspection import router as inspection_router
  from api.corrective import router as corrective_router
  from api.dashboard import router as dashboard_router

  @asynccontextmanager
  async def lifespan(app: FastAPI):
  logger.info("Starting QHSE Application...")
```

```
connection.init_db()
if not scheduler.running:
scheduler.start()
yield
logger.info("Shutting Down QHSE Application...")
scheduler.shutdown()

app = FastAPI(
title="QHSE API",
description="QHSE Backend",
version="1.0.0",
lifespan=lifespan
)

app.add_middleware(
CORSMiddleware,
allow_origins=["*"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)

app.include_router(qhse_router, prefix="/api/v1/qhse", tags=["QHSE"])
app.include_router(mvv_router, prefix="/mvv", tags=["MVV"])
app.include_router(nmobb_router, prefix="/nmobb", tags=["NMOBB"])
app.include_router(incident_router, prefix="/incident", tags=["Incident"])
app.include_router(lti_free_router, prefix="/lti-free", tags=["LTI Free"])
app.include_router(inspection_router, prefix="/inspection", tags=["Inspection"])
app.include_router(corrective_router, prefix="/corrective", tags=["Corrective"])
app.include_router(dashboard_router, prefix="/dashboard", tags=["Dashboard"])

@app.get("/health")
```

```
async def root():
return {"status": "QHSE is Running"}

if __name__ == "__main__":
uvicorn.run("main:app", host="127.0.0.1", port=8000)
```

DASHBOARD.PY

```
from sqlalchemy.orm import Session
from sqlalchemy import func, extract
from models.incident import IncidentRecord
from models.nmobb import NMRecord
from models.lti_free import LTIFreeRecord
from datetime import date, timedelta
from typing import Dict, Any

def get_leading_indicators(db: Session) -> Dict[str, Any]:
    today = date.today()
    current_year = today.year

    ytd = db.query(func.sum(NMRecord.obb_count)).filter(
        extract('year', NMRecord.week) == current_year
    ).scalar() or 0

    last_week_start = today - timedelta(days=today.weekday() + 7)
    last_week_end = last_week_start + timedelta(days=6)
    last_week = db.query(func.sum(NMRecord.obb_count)).filter(
        NMRecord.week.between(last_week_start, last_week_end)
    ).scalar() or 0
```

```
    this_week_start = today - timedelta(days=today.weekday())
    this_week_end = this_week_start + timedelta(days=6)
    this_week = db.query(func.sum(NMRecord.obb_count)).filter(
        NMRecord.week.between(this_week_start, this_week_end)
    ).scalar() or 0

    return {
        "labels": ["Observations"],
        "datasets": [
            {"label": "YTD", "data": [ytd]},
            {"label": "Last Week", "data": [last_week]},
            {"label": "This Week", "data": [this_week]},
        ]
    }

def get_lagging_indicators(db: Session) -> Dict[str, Any]:
    today = date.today()
    current_year = today.year
    last_week_start = today - timedelta(days=today.weekday() + 7)
    this_week_start = today - timedelta(days=today.weekday())

    fields = ['lti', 'rdi', 'mtc', 'fai', 'near_miss', 'fire', 'mva', 'spill',
              'property_damage', 'nwr', 'fat', 'drop_object', 'off_job_incident']

    def totals(start_date, end_date=None):
        query = db.query(*[func.sum(getattr(IncidentRecord, f)).label(f) for f in fields])
        if start_date:
            if end_date:
                query = query.filter(IncidentRecord.week.between(start_date, end_date))
            else:
                query = query.filter(IncidentRecord.week >= start_date)
        result = query.first()
```

```
        return [getattr(result, f) or 0 for f in fields]

    ytd = totals(date(current_year, 1, 1))
    last = totals(last_week_start, last_week_start + timedelta(days=6))
    this = totals(this_week_start, this_week_start + timedelta(days=6))

    return {
        "labels": [f.upper() for f in fields],
        "datasets": [
            {"label": "YTD", "data": ytd},
            {"label": "Last Week", "data": last},
            {"label": "This Week", "data": this},
        ]
    }

def get_ytd_incident_classification(db: Session) -> Dict[str, Any]:
    current_year = date.today().year
    fields = ['lti', 'rdi', 'mtc', 'fai', 'fire', 'mva', 'spill',
              'property_damage', 'nwr', 'fat', 'drop_object', 'off_job_incident']
    query = db.query(*[func.sum(getattr(IncidentRecord, f)).label(f) for f in fields])
    query = query.filter(extract('year', IncidentRecord.week) == current_year)
    result = query.first()
    data = [getattr(result, f) or 0 for f in fields]
    return {
        "labels": fields,
        "datasets": [{
            "label": "YTD Incidents",
            "data": data,
            "backgroundColor": ["#4e79a7", "#f28e2c", "#e15759", "#76b7b2", "#59a14f",
                                "#edc949", "#af7aa1", "#ff9da7", "#9c755f", "#bab0ab",
                                "#7c4dff", "#b07aa1"]
        }]
```

```
    }

def get_incident_rate(db: Session) -> Dict[str, Any]:
    # Placeholder – compute actual rates based on manhours and incident counts
    return {
        "labels": ["LTI", "TRI", "MVA", "PS Tier-1 Events"],
        "datasets": [
            {"label": "Upper Boundary", "data": [0.024, 0.06, 0.15, 0.076]},
            {"label": "SINOPEC", "data": [0.010347, 0.031042, 0.007407, 0]},
        ]
    }

def get_weekly_near_miss_analysis(db: Session) -> Dict[str, Any]:
    today = date.today()
    week_start = today - timedelta(days=today.weekday())
    week_end = week_start + timedelta(days=6)
    query = db.query(
        NMRecord.near_miss_category,
        func.sum(NMRecord.near_miss_count).label('total')
    ).filter(
        NMRecord.week.between(week_start, week_end),
        NMRecord.near_miss_category.isnot(None)
    ).group_by(NMRecord.near_miss_category).all()
    labels = [row[0] for row in query]
    data = [row[1] for row in query]
    if not labels:
        labels = ['Slip, Trip & Fall', 'Poor Situational Awareness', 'Other',
                  'Poor Housekeeping', 'Incorrect Use Of Tool', 'Pinch Point']
        data = [58, 15, 12, 6, 6, 3]
    return {
        "labels": labels,
        "datasets": [{
```

```
            "label": "Weekly Near Miss Analysis",
            "data": data,
            "backgroundColor": ["#4e79a7", "#f28e2c", "#76b7b2", "#e15759", "#59a14f",
"#edc949"]
        }]
    }

def get_obb_main_categories(db: Session) -> Dict[str, Any]:
    query = db.query(
        NMRecord.main_category,
        func.sum(NMRecord.obb_count).label('total')
    ).filter(
        NMRecord.main_category.isnot(None)
    ).group_by(NMRecord.main_category).all()
    labels = [row[0] for row in query]
    data = [row[1] for row in query]
    if not labels:
        labels = ['Procedures and House Keeping', 'Tools and Equipment',
                  'Personal Protective Equipment', 'Forklifts/Cranes/Trucks/Vehicles',
                  'Position of People', 'Other']
        data = [43, 30, 11, 7, 6, 3]
    return {
        "labels": labels,
        "datasets": [{
            "label": "OBB Main Categories",
            "data": data,
            "backgroundColor": ["#808080", "#f28e2c", "#4e79a7", "#e15759", "#76b7b2",
"#59a14f"]
        }]
    }

def get_observation_comparison(db: Session, year: int = None, week: int = None) -> Dict[str,
```

```
Any]:
    today = date.today()
    if year and week:
        last_week_start = date.fromisocalendar(year, week-1, 1) if week > 1 else date(year,1,1)
        this_week_start = date.fromisocalendar(year, week, 1)
    else:
        last_week_start = today - timedelta(days=today.weekday() + 7)
        this_week_start = today - timedelta(days=today.weekday())
    last_week_end = last_week_start + timedelta(days=6)
    this_week_end = this_week_start + timedelta(days=6)

    last_data = db.query(
        NMRecord.rig_number,
        func.sum(NMRecord.obb_count).label('total')
    ).filter(NMRecord.week.between(last_week_start,
last_week_end)).group_by(NMRecord.rig_number).all()
    this_data = db.query(
        NMRecord.rig_number,
        func.sum(NMRecord.obb_count).label('total')
    ).filter(NMRecord.week.between(this_week_start,
this_week_end)).group_by(NMRecord.rig_number).all()

    rigs = sorted(set([r[0] for r in last_data] + [r[0] for r in this_data]))
    last_dict = {r[0]: r[1] for r in last_data}
    this_dict = {r[0]: r[1] for r in this_data}

    last_week_vals = [last_dict.get(rig, 0) for rig in rigs]
    this_week_vals = [this_dict.get(rig, 0) for rig in rigs]

    return {
        "labels": rigs,
        "datasets": [
```

```
            {"label": "Last Week OBB", "data": last_week_vals, "backgroundColor": "#9ACD32"},
            {"label": "This Week OBB", "data": this_week_vals, "backgroundColor": "#CC0000"}
        ]
    }

def get_near_miss_comparison(db: Session, year: int = None, week: int = None) -> Dict[str, Any]:
    today = date.today()
    if year and week:
        last_week_start = date.fromisocalendar(year, week-1, 1) if week > 1 else date(year,1,1)
        this_week_start = date.fromisocalendar(year, week, 1)
    else:
        last_week_start = today - timedelta(days=today.weekday() + 7)
        this_week_start = today - timedelta(days=today.weekday())
    last_week_end = last_week_start + timedelta(days=6)
    this_week_end = this_week_start + timedelta(days=6)

    last_data = db.query(
        NMRecord.rig_number,
        func.sum(NMRecord.near_miss_count).label('total')
    ).filter(NMRecord.week.between(last_week_start, last_week_end)).group_by(NMRecord.rig_number).all()
    this_data = db.query(
        NMRecord.rig_number,
        func.sum(NMRecord.near_miss_count).label('total')
    ).filter(NMRecord.week.between(this_week_start, this_week_end)).group_by(NMRecord.rig_number).all()

    rigs = sorted(set([r[0] for r in last_data] + [r[0] for r in this_data]))
    last_dict = {r[0]: r[1] for r in last_data}
    this_dict = {r[0]: r[1] for r in this_data}
```

```
    last_week_vals = [last_dict.get(rig, 0) for rig in rigs]
    this_week_vals = [this_dict.get(rig, 0) for rig in rigs]

    return {
        "labels": rigs,
        "datasets": [
            {"label": "Last Week NM", "data": last_week_vals, "backgroundColor": "#9ACD32"},
            {"label": "This Week NM", "data": this_week_vals, "backgroundColor": "#CC0000"}
        ]
    }

def get_obb_analysis(db: Session) -> Dict[str, Any]:
    # Static for now; can be extended with actual categorization
    return {
        "labels": ['Positive Observation', 'Unsafe Act', 'Unsafe Condition'],
        "datasets": [{
            "data": [13, 23, 64],
            "backgroundColor": ["#4472C4", "#ED7D31", "#2E86AB"]
        }]
    }

def get_ytd_obb(db: Session) -> Dict[str, Any]:
    current_year = date.today().year
    data = db.query(
        NMRecord.rig_number,
        func.sum(NMRecord.obb_count).label('total')
    ).filter(extract('year', NMRecord.week) ==
current_year).group_by(NMRecord.rig_number).all()
    labels = [row[0] for row in data]
    values = [row[1] for row in data]
    if not labels:
        labels = ['SP-585', 'SP-41', 'SP-101', 'SP-103', 'SP-105', 'SP-106', 'SP-107',
```

```
            'SP-122', 'SP-123', 'SP-124', 'SP-125', 'SP-154', 'SP-167', 'SP-229',
            'SP-257', 'SP-258', 'SP-259', 'SP-26', 'SP-260', 'SP-261', 'SP-262',
            'SP-271', 'SP-2751', 'SP-276', 'SP-29', 'SP-30', 'SP-3046', 'SP-31',
            'SP-32', 'SP-70', 'SP-93', 'SP-96', 'SP-98', 'SP-99']
        values = [100, 6163, 2109, 2270, 1739, 1758, 1897, 1791, 812, 1, 1468, 1474,
            1, 6813, 6896, 8852, 6505, 4603, 4666, 5561, 4869, 4349, 100, 3681,
            7212, 5162, 646, 5130, 8456, 8380, 5088, 6404, 4869, 4347]
    return {
        "labels": labels,
        "datasets": [{
            "label": "YTD OBB",
            "data": values,
            "backgroundColor": "#4CAF50"
        }]
    }

def get_year_without_lti(db: Session) -> Dict[str, Any]:
    # Placeholder – compute from LTI records
    labels = ['SP-103', 'SP-101', 'SP-229', 'SP-125', 'SP-123',
            'SP-124', 'SP-165', 'SP-154', 'SP-276', 'SP-160',
            'SP-68',  'SP-122', 'SP-53L', 'SP-167', 'SP-306',
            'SP-257', 'SP-259', 'SP-262', 'SP-260', 'SP-258',
            'SP-261', 'SP-105', 'SP-161', 'SP-110', 'SP-120',
            'SP-26',  'SP-29',  'SP-31',  'SP-27',  'SP-32',
            'SP-30',  'SP-106']
    values = [19.5, 13.7, 13.1, 12.0, 11.9, 11.9, 11.7, 11.6, 10.8, 7.1,
            6.7, 6.6, 6.6, 6.3, 6.3, 5.5, 5.5, 5.1, 5.1, 5.1,
            5.0, 4.6, 4.4, 4.4, 3.6, 3.4, 3.3, 3.3, 3.3, 3.2,
            2.5, 2.3]
    return {
        "labels": labels,
        "datasets": [{
```

```
            "label": "Year without LTI",
            "data": values,
            "backgroundColor": "#4472C4"
        }]
    }

def get_ytd_near_miss_top10(db: Session) -> Dict[str, Any]:
    current_year = date.today().year
    data = db.query(
        NMRecord.rig_number,
        func.sum(NMRecord.near_miss_count).label('total')
    ).filter(extract('year', NMRecord.week) ==
current_year).group_by(NMRecord.rig_number).order_by(func.sum(NMRecord.near_miss_coun
t).desc()).limit(10).all()
    labels = [row[0] for row in data]
    values = [row[1] for row in data]
    if not labels:
        labels = ['SP-93', 'SP-260', 'SP-258', 'SP-261', 'SP-229', 'SP-103', 'SP-101', 'SP-106', 'SP-
105', 'SP-30']
        values = [417, 230, 175, 207, 202, 176, 111, 37, 81, 43]
    return {
        "labels": labels,
        "datasets": [{
            "label": "Near Misses",
            "data": values,
            "backgroundColor": "#dc2626"
        }]
    }

def get_incident_corrective_progress(db: Session) -> Dict[str, Any]:
    # Placeholder – compute from corrective actions
    labels = ['SP-103', 'SP-105', 'SP-106', 'SP-110', 'SP-120',
```

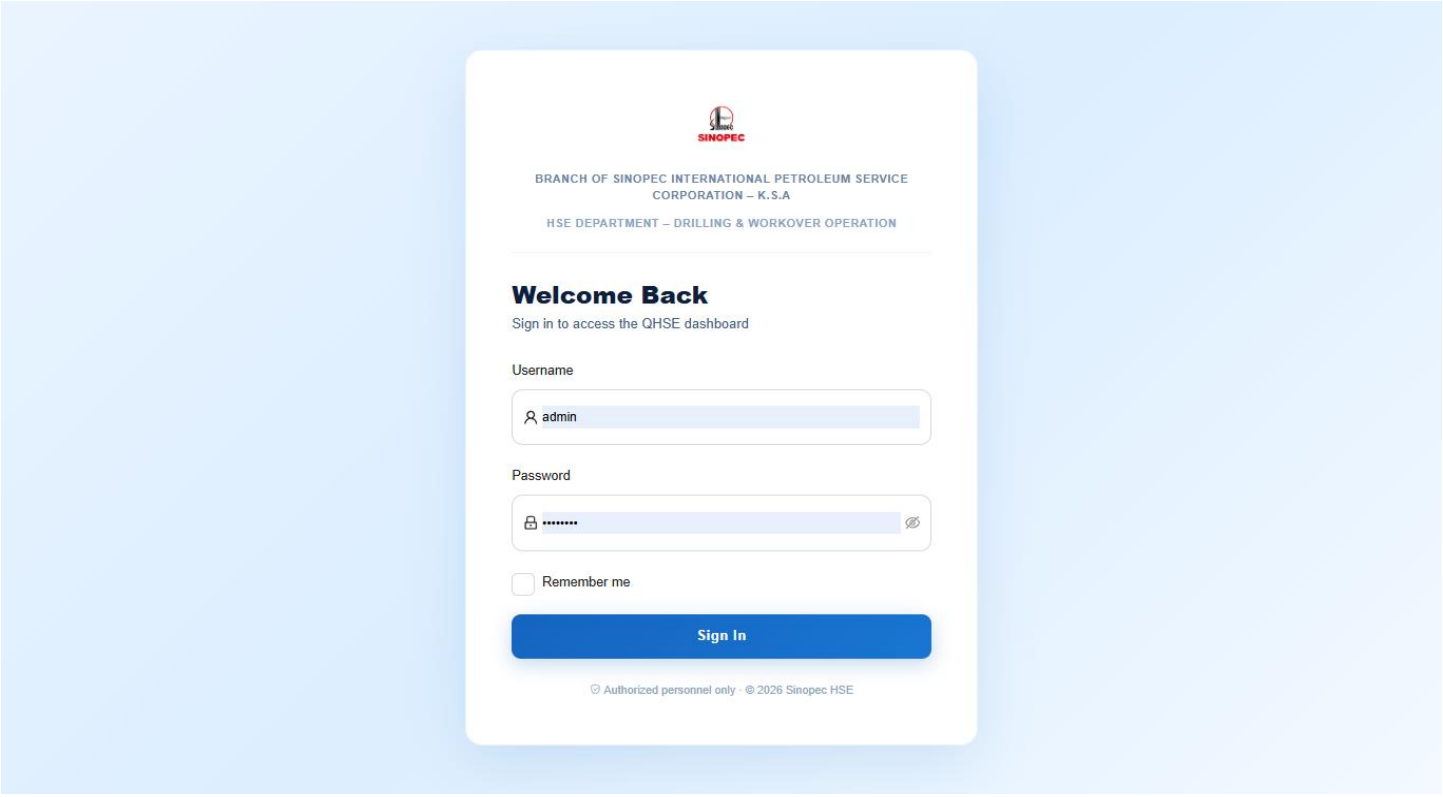
```
            'SP-121', 'SP-123', 'SP-124', 'SP-125', 'SP-154',
            'SP-160', 'SP-161', 'SP-165', 'SP-167', 'SP-275',
            'SP-101', 'SP-058', 'SP-122', 'SP-306', 'SP-257',
            'SP-259', 'SP-261', 'SP-276', 'SP-258', 'SP-260',
            'SP-262', 'SP-30',  'SP-31',  'SP-27',  'SP-26',
            'SP-29',  'SP-32',  'SP-107', 'SP-70',  'SP-93',
            'SP-96',  'SP-98',  'SP-229', 'SP-99',  'SP-053']
    values = [100, 99, 100, 63, 74, 74, 99, 100, 100, 100,
            63, 74, 74, 100, 100, 100, 100, 100, 99, 99,
            100, 100, 100, 100, 100, 100, 100, 92, 100,
            100, 100, 100, 98, 100, 100, 99, 100]
    return {
       "labels": labels,
       "datasets": [{
          "label": "Corrective Action Progress (%)",
          "data": values,
          "backgroundColor": "#5B9BD5"
       }]
    }
```

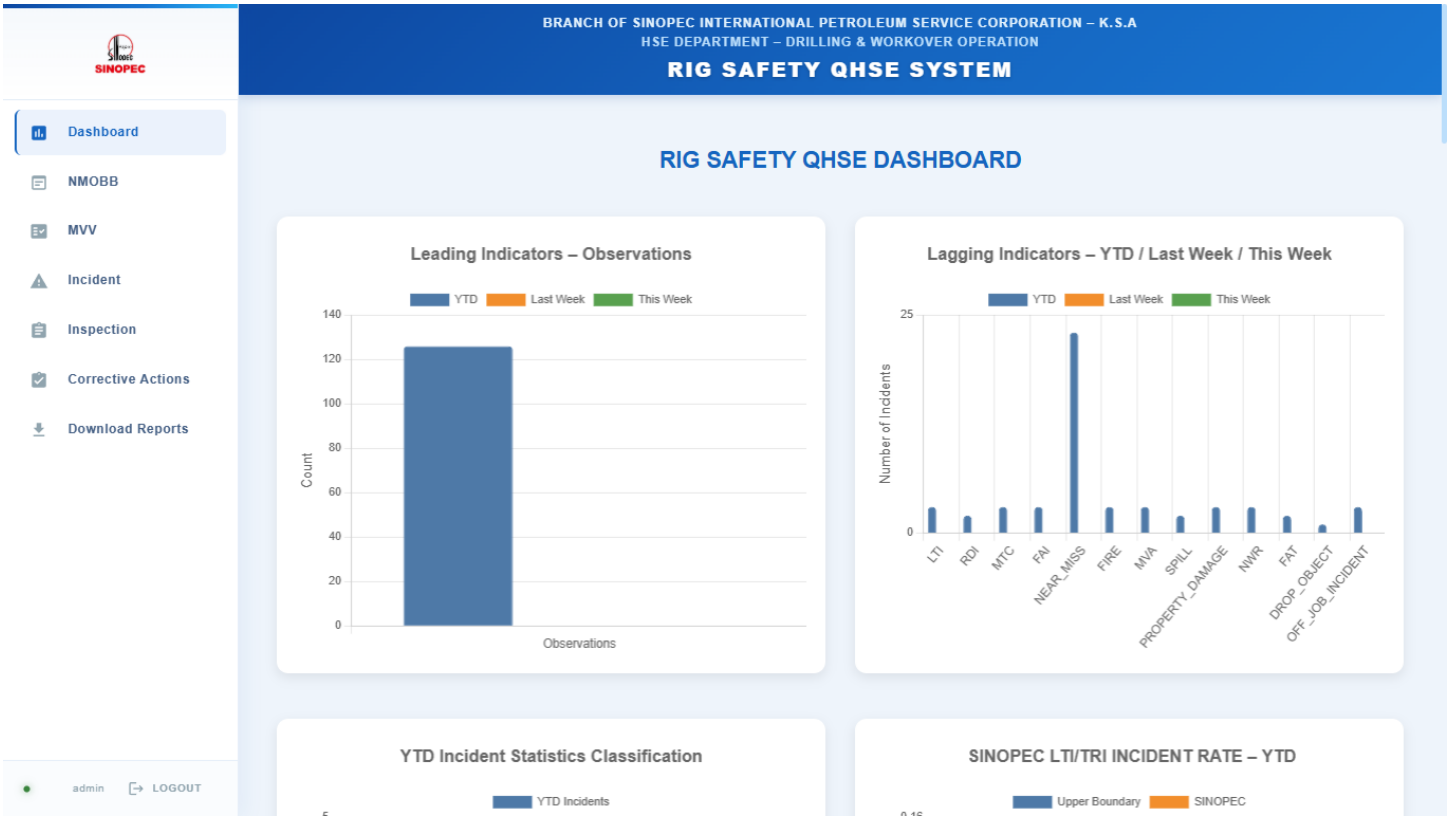
INCIDENT.PY

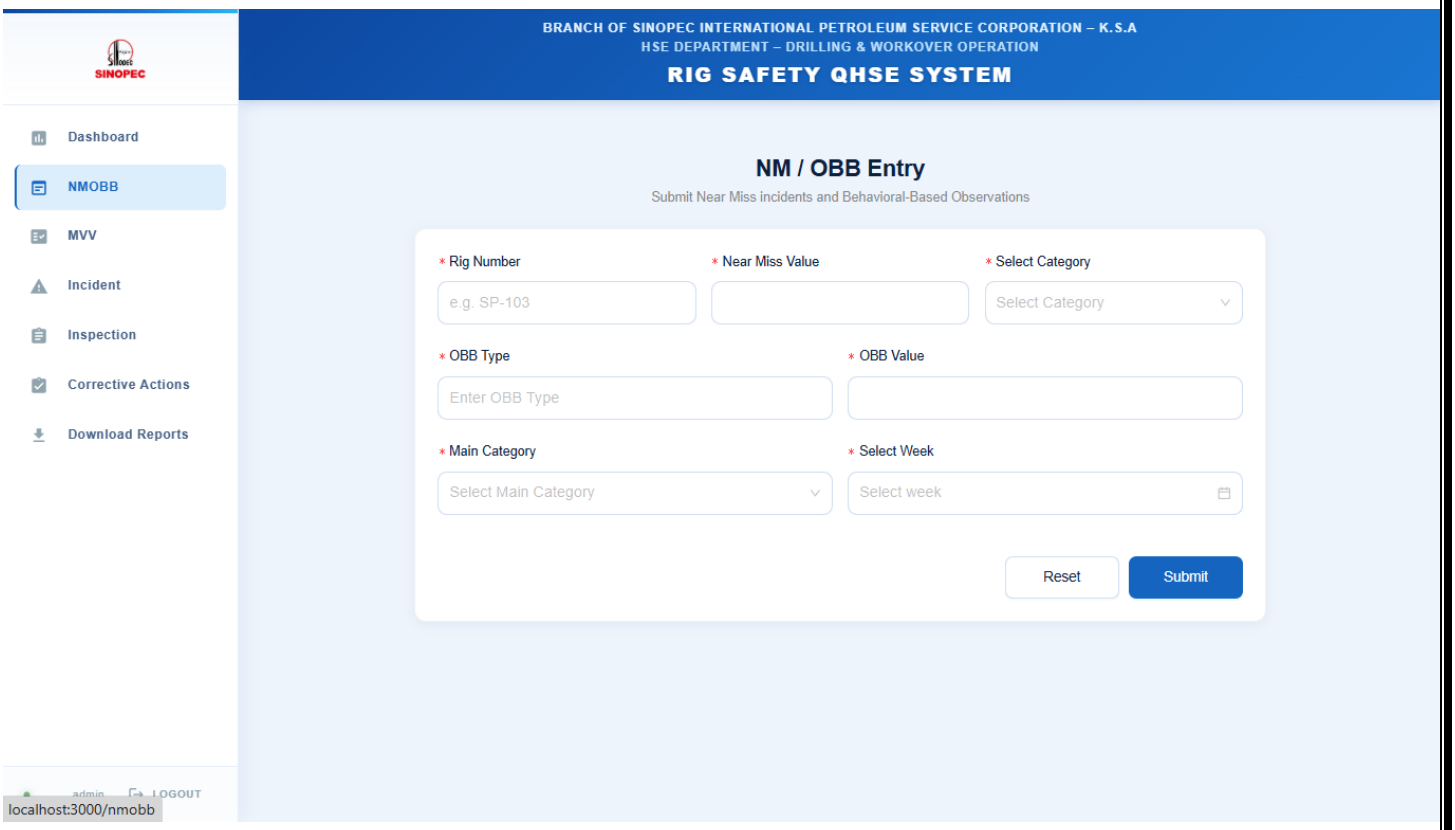
```
from sqlalchemy.orm import Session
from models.incident import IncidentRecord
from schemas.incident import IncidentCreate
from typing import List, Optional

def create_incident(db: Session, incident: IncidentCreate) -> IncidentRecord:
   db_incident = IncidentRecord(**incident.dict())
   db.add(db_incident)
   db.commit()
```

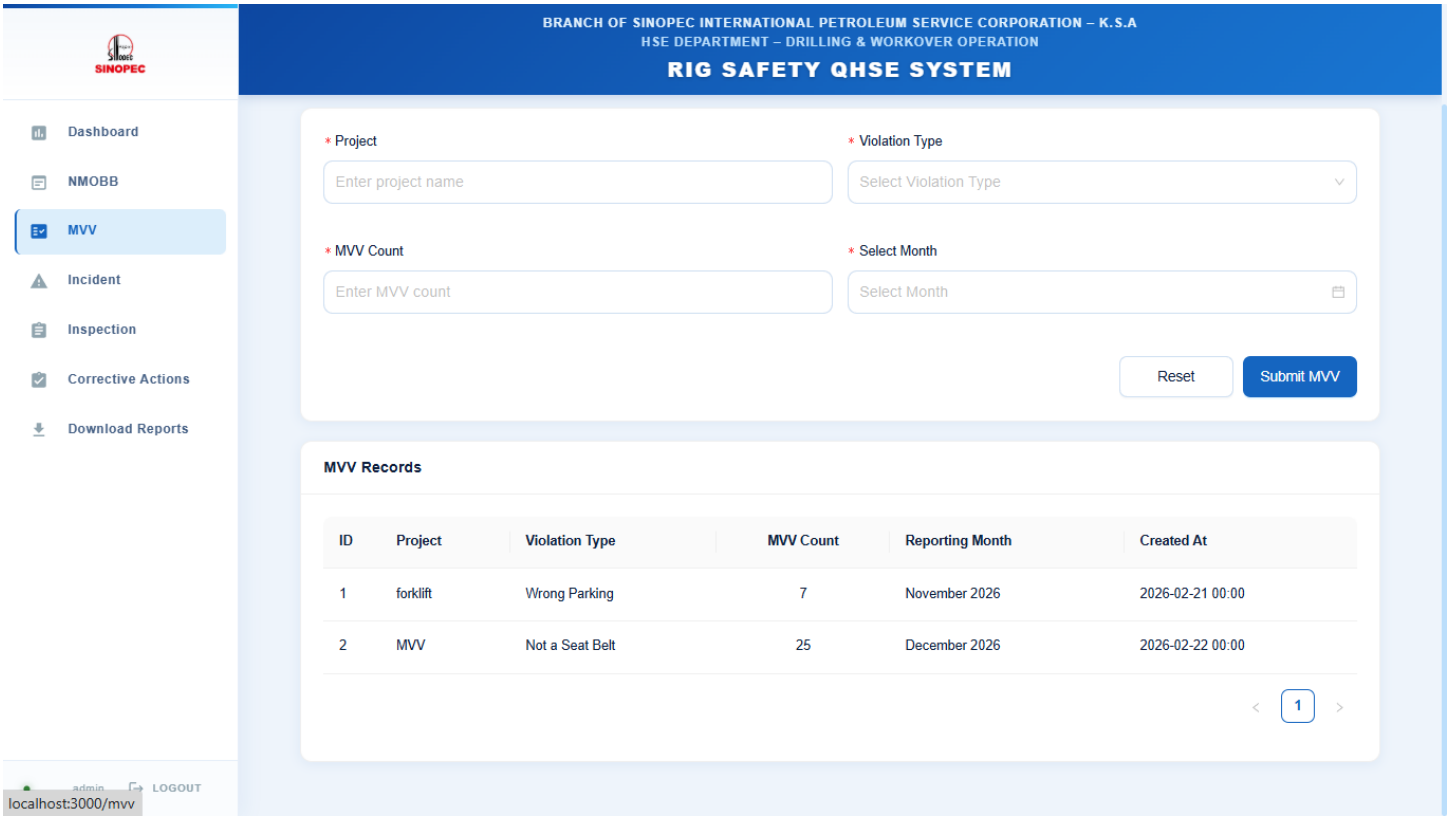
```
    db.refresh(db_incident)
    return db_incident

def get_all_incidents(db: Session) -> List[IncidentRecord]:
    return db.query(IncidentRecord).all()

def get_incident(db: Session, incident_id: int) -> Optional[IncidentRecord]:
    return db.query(IncidentRecord).filter(IncidentRecord.id == incident_id).first()

def update_incident(db: Session, incident_id: int, incident: IncidentCreate) ->
Optional[IncidentRecord]:
    db_incident = get_incident(db, incident_id)
    if db_incident:
        for key, value in incident.dict().items():
            setattr(db_incident, key, value)
        db.commit()
        db.refresh(db_incident)
    return db_incident

def delete_incident(db: Session, incident_id: int) -> bool:
    db_incident = get_incident(db, incident_id)
    if db_incident:
        db.delete(db_incident)
        db.commit()
        return True
    return False
```

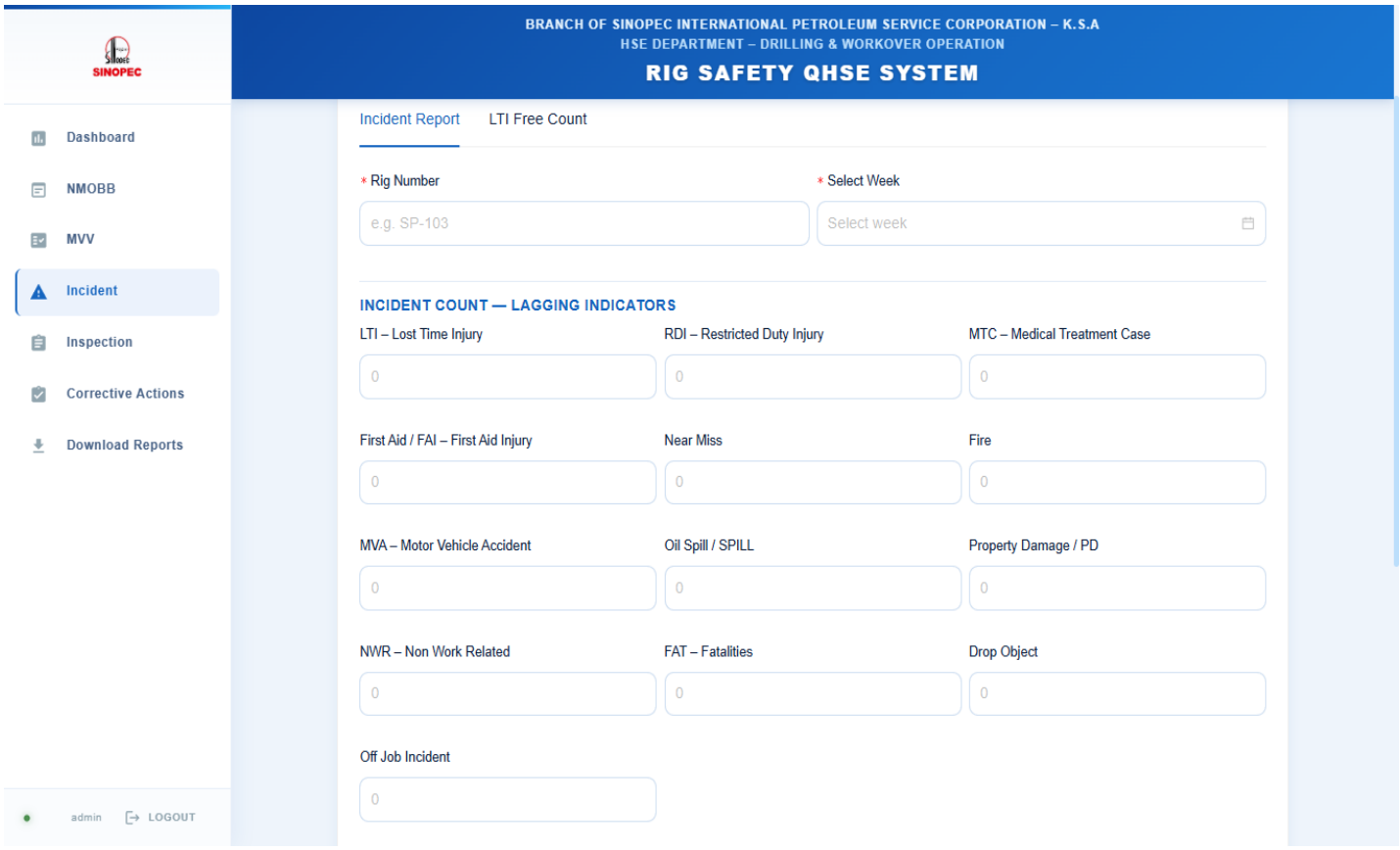
OUTPUT SCREEN

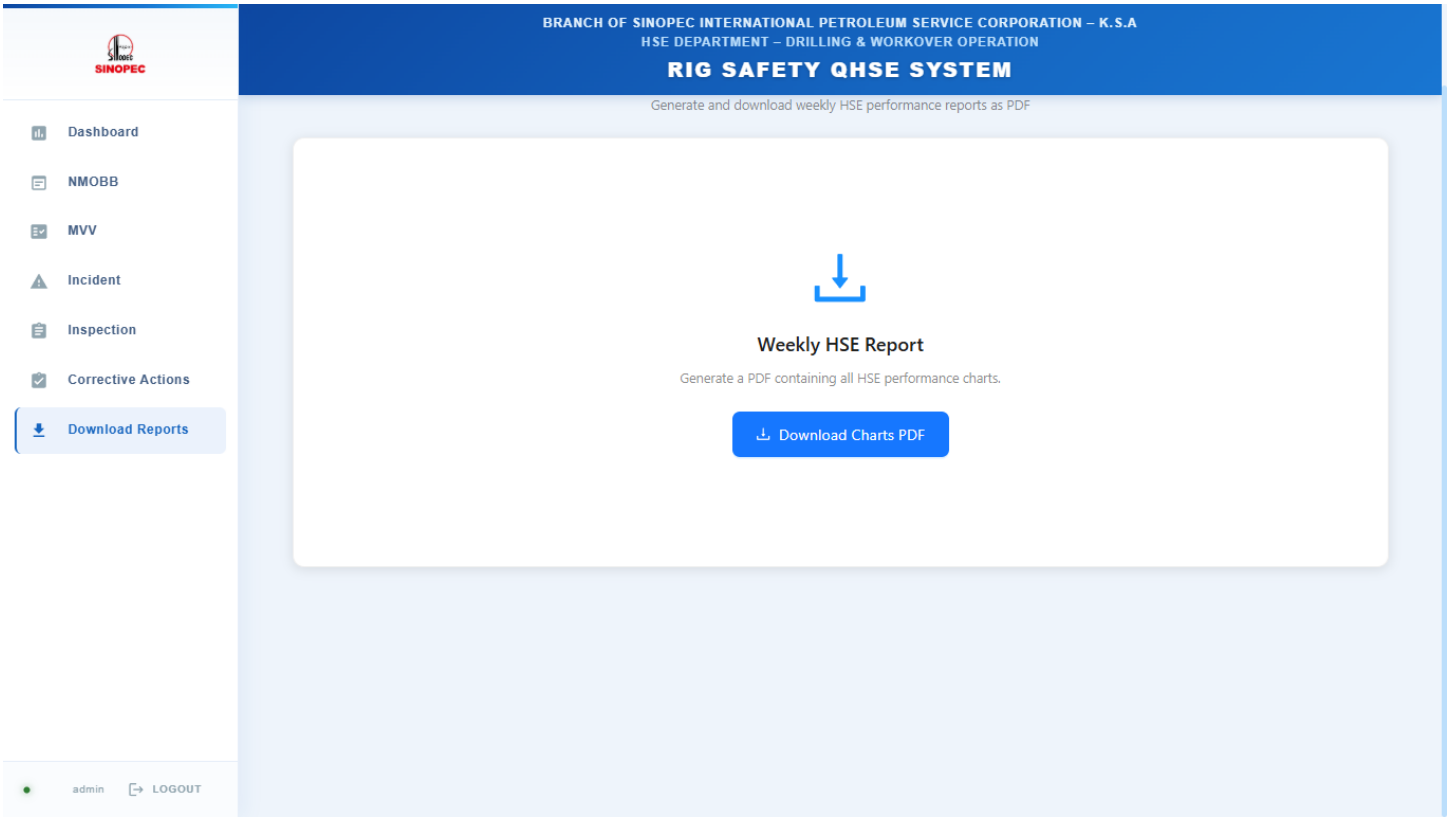





BRANCH OF SINOPEC INTERNATIONAL PETROLEUM SERVICE CORPORATION – K.S.A
HSE DEPARTMENT – DRILLING & WORKOVER OPERATION
RIG SAFETY QHSE SYSTEM
Dashboard
NMOBB
MVV
Incident
Inspection
Corrective Actions
Download Reports
NM / OBB Entry
Submit Near Miss incidents and Behavioral-Based Observations
* Rig Number
e.g. SP-103
* Near Miss Value
* Select Category
Select Category
* OBB Type
Enter OBB Type
* OBB Value
* Main Category
Select Main Category
* Select Week
Select week
Reset
Submit
admin
LOGOUT
localhost:3000/nmobb


BRANCH OF SINOPEC INTERNATIONAL PETROLEUM SERVICE CORPORATION – K.S.A
HSE DEPARTMENT – DRILLING & WORKOVER OPERATION
RIG SAFETY QHSE SYSTEM
Dashboard
NMOBB
MVV
Incident
Inspection
Corrective Actions
Download Reports
* Project
Enter project name
* Violation Type
Select Violation Type
* MVV Count
Enter MVV count
* Select Month
Select Month
Reset
Submit MVV
MVV Records
ID
Project
Violation Type
MVV Count
Reporting Month
Created At
1
forklift
Wrong Parking
7
November 2026
2026-02-21 00:00
2
MVV
Not a Seat Belt
25
December 2026
2026-02-22 00:00
1
admin
LOGOUT
localhost:3000/mvv


SINOPEC
Dashboard
NMOBB
MVV
Incident
Inspection
Corrective Actions
Download Reports
admin
LOGOUT
BRANCH OF SINOPEC INTERNATIONAL PETROLEUM SERVICE CORPORATION – K.S.A
HSE DEPARTMENT – DRILLING & WORKOVER OPERATION
RIG SAFETY QHSE SYSTEM
Incident Report
LTI Free Count
* Rig Number
e.g. SP-103
* Select Week
Select week
INCIDENT COUNT — LAGGING INDICATORS
LTI – Lost Time Injury
RDI – Restricted Duty Injury
MTC – Medical Treatment Case
First Aid / FAI – First Aid Injury
Near Miss
Fire
MVA – Motor Vehicle Accident
Oil Spill / SPILL
Property Damage / PD
NWR – Non Work Related
FAT – Fatalities
Drop Object
Off Job Incident


SINOPEC
Dashboard
NMOBB
MVV
Incident
Inspection
Corrective Actions
Download Reports
admin
LOGOUT
BRANCH OF SINOPEC INTERNATIONAL PETROLEUM SERVICE CORPORATION – K.S.A
HSE DEPARTMENT – DRILLING & WORKOVER OPERATION
RIG SAFETY QHSE SYSTEM
Generate and download weekly HSE performance reports as PDF
Weekly HSE Report
Generate a PDF containing all HSE performance charts.
Download Charts PDF

CONCLUTION

The QHSE Dashboard Backend successfully provides a structured and efficient system for managing safety and compliance data within an organization. By utilizing modern technologies like FastAPI and SQLAlchemy, the project ensures scalability, reliability, and effective data handling. It enables the monitoring of key safety metrics and supports informed decision-making. Overall, the system enhances workplace safety, improves operational efficiency, and promotes continuous improvement in QHSE practices.

FUTURE ENHANCEMENT

- Implement cloud-based deployment for better scalability and remote access
- Add real-time data updates using Web Sockets
- Integrate machine learning for risk prediction and data analysis
- Introduce role-based access control (RBAC) for security
- Develop a mobile application for easy access
- Integrate IoT devices for automated data collection
- Enhance data visualization with interactive charts and reports
- Support multiple databases (e.g., PostgreSQL, MySQL)
- Add notification system (email/SMS alerts)
- Improve UI/UX design for better user experience
- Include multi-language support

REFERENCES

- https://www.php.net/manual/en/
- https://www.w3schools.com/
- https://developer.mozilla.org/
- https://stackoverflow.com/
- https://dev.mysql.com/doc/